

# Git Practical Assignment

## Rebase, Reset, Revert, Restore, Stash, Merge and Conflict Resolution

Objective: Complete the following practical tasks in a Git repository. Students must use the appropriate Git command for each situation and show the Git history/status before and after important operations.

### General Instructions

- Create a repository named git-practical-assignment.
- Use meaningful commit messages for every required commit.
- Run git status and git log --oneline --graph --all whenever required.
- Do not use a command simply because it is listed in the task; choose the command that fits the situation.
- Take screenshots of important results for submission.

### Task 1 – Git Branching and Merge

Scenario: Two developers are working on different parts of the ShopEasy website. The main branch contains the basic website, while a feature branch contains a new product page.

1. Create index.html and commit it with the message: Initial website.
2. Create a branch named feature-product.
3. On feature-product, create products.html and commit it with: Added product page.
4. Switch to main.
5. Make another change to index.html and commit it with: Updated homepage.
6. Merge feature-product into main.
7. Verify the merge using git log --oneline --graph --all.

```
root@ip-172-31-29-185:~/git-practical-assignment — ssh -i one-key.pem ec2-user@3.93.46.244
162 mkdir git-practical-assignment
163 cd git-practical-assignment
164 git init
165 echo "welcome to Shop-Easy" > index.html
166 git add index.html
167 git commit -m "Initial website"
168 git branch feature-product
169 git switch feature-product
170 git branch --show current
171 echo "Products" > products.html
172 git add products.html
173 git commit -m "Added product page"
174 git checkout master
175 git merge feature-product
176 git log --oneline --graph --all
177 history
[root@git git-practical-assignment]#

root@ip-172-31-29-185:~/git-practical-assignment — ssh -i one-key.pem ec2-user@3.93.4...
[ root@git git-practical-assignment]# git log --oneline --graph --all
* d122b9b (HEAD -> master, feature-product) Added product page
* 8a4d1a8 Initial website
[ root@git git-practical-assignment]#
```

Expected concept: Changes from the feature branch should become part of main.

## Task 2 – Merge Conflict

Scenario: Two developers changed the same line of index.html. Git cannot automatically decide which version should be kept.

8. Create a branch named developer1 and modify the same line in index.html.
9. Commit the change.
10. Switch to main and modify that same line differently.
11. Commit the change.
12. Merge developer1 into main.
13. When Git reports a conflict, inspect index.html and identify the conflict markers.
14. Manually edit the file and keep the correct final content.
15. Run git add index.html.
16. Complete the merge commit.
17. Verify the final history and file content.

```
[root@git git-practical-assignment]# git log --oneline --graph --all
* 7616cb8 (HEAD -> master) merge-conflict
| * eb81ed2 (developer-1) developer-1 changes
|/
* d122b9b (feature-product) Added product page
* 8a4d1a8 Initial website
[ root@git git-practical-assignment]# history
```

Students must explain why the conflict happened and how it was resolved.

```
182 #TASK-2
183 git checkout master
184 git branch developer-1
185 git checkout developer-1
186 cat index.html
187 vim index.html
188 git add index.html
189 git commit -m "developer-1 changes" index.html
190 git checkout master
191 vim index.html
192 git commit -m "merge-conflict"
193 cat index.html
194 vim index.html
195 git add index.html
196 git commit -m "merge-conflict"
197 git status
198 git merge developer-1
199 git log --oneline --graph --all
200 clear
201 git log --oneline --graph --all
202 history
[root@git git-practical-assignment]#
```

## Task 3 – Git Rebase

Scenario: A developer is working on a feature branch while new commits are added to main. The developer wants the feature branch to be based on the latest main history.

18. Create a branch named feature-login from main.
19. Make two commits on feature-login.
20. Switch to main and make two new commits.
21. Switch back to feature-login.
22. Rebase feature-login onto main.
23. Use git log --oneline --graph --all to inspect the history.

```
Successfully rebased and updated refs/heads/feature-login.
[root@git git-practical-assignment]# git log --oneline --graph --all
* 990f3c8 (HEAD -> feature-login) adding login-2
* 5404845 adding logic
* 443d9e7 (master) committing f2
* 6bf0dec committing f1
* 5ec89c4 merge-conflict
| \
| * eb81ed2 (developer-1) developer-1 changes
* | 7616cb8 merge-conflict
| /
* d122b9b (feature-product) Added product page
* 8a4d1a8 Initial website
[root@git git-practical-assignment]#
```

```

211 #TASK-3
212 CLEAR
213 clear
214 git branch feature-login
215 git switch feature-login
216 touch login
217 git add login
218 git commit -m "adding logic" logic
219 git commit -m "adding logic" login
220 touch login-2
221 git add login-2
222 git commit -m "adding login-2" login-2
223 git checkout master
224 touch f1
225 git add f1
226 git commit -m "committing f1" f1
227 touch f2
228 git add f2
229 git commit -m "committing f2" f2
230 git checkout feature-login
231 git rebase master
232 git log --oneline --graph --all

```

Expected concept: Rebase replays the feature commits on top of the updated main branch.

## Task 4 – Interactive Rebase

Scenario: The feature branch contains several small commits such as Fix button, Fix button again, and Final button fix. The developer wants a cleaner local history.

24. Create at least four local commits.
25. Run `git rebase -i HEAD~4`.
26. Use pick and squash/fixup where appropriate to combine unnecessary commits.
27. Verify the

```

+ root@ip-172-31-29-185:~/git-practical-assignment — ssh -i one-key.pem ec2-user@3.93.46.244
~/Downloads/keys
[feature-login cde5581] committing IR-3
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 IR-3
[root@git git-practical-assignment]# git commit -m "committing IR-4" IR-4
[feature-login 6d4ef7a] committing IR-4
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 IR-4
[root@git git-practical-assignment]# git log --oneline
6d4ef7a (HEAD -> feature-login) committing IR-4
cde5581 committing IR-3
e45d256 committing IR-2
aedb45e committing IR-1
990f3c8 (interactive-rebase) adding login-2
5404845 adding logic
443d9e7 (master) committing f2
6bf0dec committing f1
5ec89c4 merge-conflict
7616cb8 merge-conflict
eb81ed2 (developer-1) developer-1 changes
d122b9b (feature-product) Added product page
8a4d1a8 Initial website
[root@git git-practical-assignment]#

```

cleaned history using `git log --oneline`.

Do this only on local commits that are safe to rewrite.

```

235 #TASK-4
236 git branch interactive-rebase
237 touch IR-1 IR-2 IR-3 IR-4
238 git add *
239 git commit -m "committing IR-1" IR-1
240 git commit -m "committing IR-2" IR-2
241 git commit -m "committing IR-3" IR-3
242 git commit -m "committing IR-4" IR-4
243 git log --oneline
244 history
[root@git git-practical-assignment]#

```

## Task 5 – Git Reset

Scenario: A developer accidentally created a commit locally and wants to move the branch back.

28. Create and commit a file named test.txt.
29. Make another commit named Unwanted change.
30. Use git reset --soft HEAD~1 and inspect git status.
31. Repeat the experiment in a safe practice branch using git reset --mixed HEAD~1.
32. Create another test commit and demonstrate git reset --hard HEAD~1.
33. For each reset type, record what happens to HEAD, the staging area, and the working directory.

Warning: Do not use --hard on important work.

```
root@ip-172-31-29-185:~/git-practical-assignment -- ssh -i one-key.pem ec2-user@3.93.46.244
+
246 #TASK-5 GIT RESET
247 git branch reset
248 git switch reset
249 cat > test.txt
250 cat test.txt
251 git add test.txt
252 git commit -m "committing test" test.txt
253 vim test.txt
254 git add test.txt
255 git commit -m "useless addition :)" test.txt
256 git reset --soft HEAD~1
257 git reset --soft
258 git status
259 git add test.txt
260 git commit -m "useless change :)" test.txt
261 git reset --soft HEAD~1
262 git reset --soft
263 git status
264 git add test.txt
265 git commit -m "useless change :)" test.txt
266 git reset --mixed HEAD~1
267 git status
268 git add test.txt
269 git commit -m "jst another commit:)" test.txt
270 cat >> test.txt
271 git add test.txt
272 git commit -m "jst another commit again:)" test.txt
273 git reset --hard HEAD~1
274 git status
275
```

## Task 6 – Restore

Scenario: A developer changed a file but wants to discard the local modification, and later accidentally staged a file.

- |                                                                                                       |                                                       |
|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| 34. Modify index.html without committing.                                                             | 281 #TASK-6 : RESTORE                                 |
| 35. Use git restore index.html to discard the working-directory change.                               | 282 echo "just a temporary change" >> index.html      |
| 36. Modify index.html again and stage it using git add.                                               | 283 git status                                        |
| 37. Use git restore --staged index.html to remove it from the staging area without deleting the file. | 284 git restore index.html                            |
| 38. Verify the result using git status.                                                               | 285 git status                                        |
|                                                                                                       | 286 echo "yet another temporary change" >> index.html |
|                                                                                                       | 287 git add index.html                                |
|                                                                                                       | 288 git status                                        |
|                                                                                                       | 289 git restore --staged index.html                   |
|                                                                                                       | 290 git status                                        |
|                                                                                                       | 291 history                                           |
|                                                                                                       | [root@qit git-practical-assignment]#                  |

## Task 7 – Stash

Scenario: A developer is halfway through a feature when an urgent production fix is requested. The unfinished work should be saved without creating an incomplete commit.

39. Modify index.html and style.css.
40. Run git status.
41. Save the unfinished changes using git stash.

42. Verify that the working directory is clean.
  43. Create a separate urgent-fix change and commit it.
  44. Return to the feature work.
  45. View the stash using git stash list.
  46. Recover the work using git stash pop.
  47. Verify that the previous changes are back.
- Extension: Create two stashes and demonstrate git stash apply stash@{1}.

```

295 #TASK-7 : STASH
296 git branch stash
297 ls
298 cd git-practical-assignment/
299 clear
300 git init
301 git branch stash
302 git switch stash
303 git branch --show current
304 cat > stash.txt
305 ls
306 rm stash.txt
307 ls
308 cat >> index.html
309 cat > style.css
310 git status
311 git stash
312 git status
313 cat >> index.html
314 git commit -m "last-moment changes"
315 git branch master
316 git checkout master
317 cat >> index.html
318 git add index.html
319 git commit -m "last-moment changes"
320 git stash list
321 git stash pop
322 git status

```

## Task 8 – Git Revert

Scenario: A website change has already been committed and pushed. The change is found to be incorrect, but the existing history should not be rewritten.

48. Create a commit named Website Version 1.
49. Create a second commit named Website Version 2.
50. Pretend that Version 2 has already been shared with the team.
51. Use git revert HEAD.
52. Verify that Git created a new commit that reverses the previous change.
53. Show the history using git log --oneline.

Students must explain why revert is preferred over reset for an already-shared commit.

```

[root@git git-practical-assignment]# git commit -m "modified-version.txt"
[master dc3746f] modified-version.txt
3 files changed, 8 insertions(+), 1 deletion(-)
create mode 100644 style.css
create mode 100644 version.txt
[root@git git-practical-assignment]# echo "new-web1" > version2.txt
[root@git git-practical-assignment]# git add version2.txt
[root@git git-practical-assignment]# git commit -m "modified-version2.txt" version2.txt

[master b33477c] modified-version2.txt
1 file changed, 1 insertion(+)
create mode 100644 version2.txt
[root@git git-practical-assignment]# git revert HEAD
[master 1ef013b] Revert "modified-version2.txt"
1 file changed, 1 deletion(-)
delete mode 100644 version2.txt
[root@git git-practical-assignment]# git log --oneline
1ef013b (HEAD -> master) Revert "modified-version2.txt"
b33477c modified-version2.txt
dc3746f modified-version.txt
6ec5aed last-moment changes
443d9e7 (stash) committing f2
6bf0dec committing f1
5ec89c4 merge-conflict
7616cb8 merge-conflict
eb81ed2 (developer-1) developer-1 changes
d122b9b (feature-product) Added product page
8a4d1a8 Initial website
[root@git git-practical-assignment]# █

```

## Task 9 – Combined Real-World Challenge

Scenario: You are a developer working on ShopEasy. You are using a feature branch while another developer updates main. During your work, you are interrupted by an urgent production fix.

54. Create a feature branch and make two unfinished changes.

55. Stash the unfinished work.
56. Switch to main and create an urgent-fix commit.
57. Return to the feature branch and recover the stash.
58. Update the feature and create two commits.
59. Bring the latest main changes into the feature branch using rebase.
60. Create a deliberate conflict with a main-branch change and resolve it.
61. Create a final commit.
62. Assume the final commit is wrong and use git revert to undo it.
63. Use interactive rebase on safe local commits to clean up the remaining feature history.
64. Show the final history using git log --oneline --graph --all.

```

root@git:/git-practical-assignment -- ssh -i one-key.pem ec2-user@3.93.46.244
~/Downloads/keys
1 file changed, 1 deletion(-)
Successfully rebased and updated refs/heads/combined-feature.
* d3ca8f6 (HEAD -> combined-feature) Revert "Final feature change"
* 6c27cfc Final feature change
* c82c310 Feature update 2
* ad4ee01 Feature update 1
* f4b6db9 Urgent fix
| * 75a3dff (refs/stash) WIP on combined-feature: 1ef013b Revert "modified-version2.txt"
|/
| * 4631b2e index on combined-feature: 1ef013b Revert "modified-version2.txt"
|/
* 1ef013b (master) Revert "modified-version2.txt"
* b33477c modified-version2.txt
* dc3746f modified-version.txt
* 6ec5aed last-moment changes
| * 232e4ed (reset) jst another commit:)
| * b496d8f committing test
| * 6d4ef7a (feature-login) committing IR-4
| * cde5581 committing IR-3
| * e45d256 committing IR-2
| * aedb45e committing IR-1
| * 990f3c8 (interactive-rebase) adding login-2
| * 5404845 adding logic
|/
* 443d9e7 (stash) committing f2
* 6bf0dec committing f1
* 5ec89c4 merge-conflict
| \
| * eb81ed2 (developer-1) developer-1 changes
| * 7616cb8 merge-conflict
[root@git git-practical-assignment]#

```

Final expected flow: Branch → Stash → Fix → Recover Stash → Commit → Rebase → Resolve Conflict → Revert → Interactive Rebase.

## Task 10 – Command Selection Challenge

For each situation, write the Git command you would choose and explain why.

| Situation                                                    | Command                          | Why? |
|--------------------------------------------------------------|----------------------------------|------|
| Remove the latest local commit but keep its changes staged.  | Git reset – soft HEAD~1          |      |
| Discard an uncommitted modification to a file.               | Git restore <file-name>          |      |
| Undo a commit that has already been shared.                  | Git revert HEAD                  |      |
| Temporarily save unfinished work.                            | git stash                        |      |
| Move feature commits on top of the latest main branch.       | git rebase main                  |      |
| Combine several local commits.                               | git rebase -i HEAD~4             |      |
| Remove a file from staging but keep the file.                | git restore --staged <file-name> |      |
| Combine changes from another branch into the current branch. | git merge branch-name            |      |

## Submission Requirements

- Git repository containing the complete practical work.

- Screenshots of branch history before and after merge/rebase.
- Screenshot showing a merge conflict and its resolution.
- Screenshots showing reset, restore, stash, and revert results.
- Final git log --oneline --graph --all output.
- A short explanation of why each command was selected.

## Quick Reference

| Command     | Main Purpose                                     |
|-------------|--------------------------------------------------|
| git merge   | Combine changes from another branch              |
| git rebase  | Replay commits on a new base                     |
| git reset   | Move HEAD and change local staging/working state |
| git revert  | Undo a commit by creating a new commit           |
| git restore | Restore file or staging state                    |
| git stash   | Temporarily save unfinished changes              |